



Spotfire Copilot™ Installation Guide

Part 1 - Backend

1. Introduction.....	3
1.1 Purpose of this Document.....	3
Key Components.....	3
Important Considerations.....	3
Tailoring for Cloud Platforms.....	3
1.2 Recommended Personnel.....	4
1.3 Prerequisites.....	4
2. Setting Up Backend Infrastructure.....	5
Core Components.....	5
Deployment Flexibility.....	5
Integration Points.....	5
Scalability and Performance.....	6
Security Considerations.....	6
2.1 Setup Contextual Documents.....	7
2.1.1 Setup Data Loaders.....	7
2.1.2 Available Loaders.....	7
2.1.3 Steps to set up Data Loaders:.....	8
2.2 Setup Orchestrator.....	10
2.2.1 Supported Integrations.....	11
2.2.2 Steps to set up the Orchestrator:.....	11
2.3 Authentication & Access Guide.....	12
2.3.1 Swagger UI Interface.....	13
2.3.2 Direct REST API Access (Programmatic Integration).....	14
APPENDIX.....	17
Docker Compose files.....	17
Data Loaders.....	18
Environment Variables.....	18
Basic Data Loader - docker-compose-basic-data-loader.yml.....	22
Advanced Data Loader - public.ecr.aws/tds/data-loader-pdf-unstruct:latest.....	23
Orchestrator.....	24
Environment Variables.....	24
Orchestrator - docker-compose-orchestrator.yml.....	28
Docker Containers - Security Guidelines.....	28

1. Introduction

1.1 Purpose of this Document

This document provides comprehensive instructions for setting up the Spotfire Copilot™ backend, with a focus on local deployments. It covers:

Key Components

- Deployment of Docker containers
- Establishment of a robust foundation for the Copilot project
- Guidance for both new users and those optimizing existing deployments

Important Considerations

This guide primarily explains local deployment scenarios. While the core concepts remain applicable, it's crucial to understand that cloud-based deployments on platforms like AWS, Azure, or Google Cloud will require significant adaptations:

- **Infrastructure as Code:** Cloud deployments often utilize platform-specific tools for infrastructure provisioning and management.
- **Networking:** Cloud environments have unique networking configurations and security groups that differ from local setups.
- **Scalability:** Cloud platforms offer native scaling options that aren't covered in local deployment instructions.
- **Managed Services:** Cloud providers may offer managed alternatives to some components described here.

Tailoring for Cloud Platforms

When adapting these instructions for cloud deployments:

- Consult the specific cloud provider's documentation for best practices.
- Consider using cloud-native container orchestration services (e.g., EKS, AKS, GKE) instead of managing Docker containers directly.
- Implement appropriate cloud security measures and access controls.
- Leverage cloud-specific monitoring and logging solutions.

This guide serves as a starting point for understanding the Copilot backend architecture. However, cloud deployments will require additional expertise and platform-specific considerations to ensure optimal performance, security, and scalability.

1.2 Recommended Personnel

We recommend involving the following persona from your organization to set up the backend infrastructure:

IT/Analytics Engineer:

- Familiar with installing containerized applications locally and in the cloud
- Skilled in Docker and Docker Compose for multi-container setups
- Solid understanding of system configurations
- Experience navigating cloud platforms (for remote deployments)

1.3 Prerequisites

Before beginning the setup process, ensure you have the following:

1. Access to an LLM and Embedding models from one of these providers:

- OpenAI
- Azure OpenAI
- AWS Bedrock
- Google Gemini
- Cohere
- Hugging Face
- Ollama
- Nvidia NIM

Note: For other model providers, contact datascience@spotfire.com for assistance.



2. Access to a VectorDB from one of these providers:

- Milvus
- Zilliz
- Azure Cognitive Search
- Redis
- MongoDB
- Chroma
- Qdrant
- VertexAI Vector Search

Note: For other VectorDB providers, contact datascience@spotfire.com for assistance.

3. A secret key for signing JWT tokens:

- Generate using OpenSSL: `openssl rand -hex 32`

4. A Bcrypt Hashed Password for the "admin" user:

- Generate online at <https://bcrypt-generator.com/>
- This password will be used for:
 - i. Authorizing access to administrative functions
 - ii. Generating JWT (JSON Web Token) tokens for authentication
- Important: Save the original password as it's needed to obtain the JWT token

5. Docker installed for deploying backend components as Docker Compose applications

2. Setting Up Backend Infrastructure

The Copilot backend consists of several key components deployed as Docker containers, allowing for flexible deployment options in both on-premise and hybrid cloud environments. This architecture enables seamless integration with various data sources, vector databases, and large language models (LLMs).

Core Components

- **Data Loaders:** Containerized applications responsible for ingesting and processing data from diverse sources and loading into Vector Databases.
- **Orchestrator:** Containerized application that manages the workflow and coordinates interactions between various system elements.
- **Vector Database:** An on-premise or cloud-hosted solution for efficient storage and retrieval of high-dimensional vectors, crucial for semantic search capabilities.
- **Large Language Model (LLM):** Either self-hosted on powerful on-premise hardware or accessed via API from cloud providers.

Deployment Flexibility

This architecture supports various deployment scenarios:

- **Fully On-Premise:** All components, including VectorDBs and LLMs, are hosted within the organization's infrastructure.
- **Hybrid Cloud:** Core Docker components (Data Loaders and Orchestrator) run on-premise, while leveraging cloud-based VectorDBs or LLM APIs for enhanced performance and scalability.
- **Multi-Cloud:** Distributed deployment across multiple cloud providers for redundancy and optimal resource utilization.

Integration Points

- **Data Sources:** The Data Loaders connect to various internal and external data repositories, ensuring comprehensive information gathering.
- **VectorDB Interface:** The Orchestrator communicates with the VectorDB for efficient storage and retrieval of processed data embeddings.
- **LLM Integration:** Seamless interaction between the Orchestrator and the chosen LLM, whether on-premise or cloud-based, for advanced natural language processing tasks.

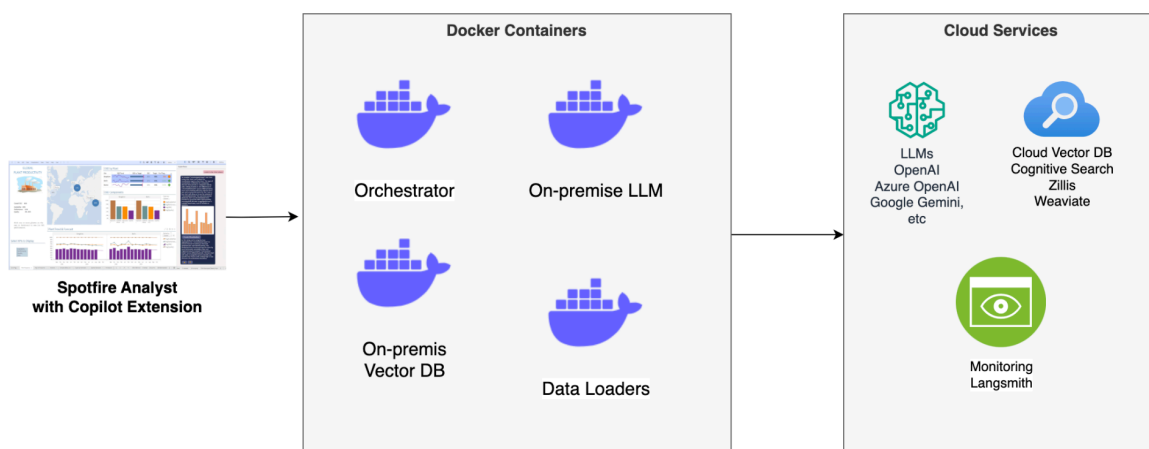
Scalability and Performance

- Docker containers enable easy scaling of Data Loaders and Orchestrator components based on workload demands.
- VectorDBs can be scaled independently to handle growing data volumes and query loads.
- LLM processing can be distributed across multiple nodes or leveraged from cloud providers to balance performance and cost.

Security Considerations

- Network isolation and secure communication protocols between components.
- Data encryption at rest and in transit, especially crucial for sensitive information in on-premise deployments.
- Access control and authentication mechanisms for all system components.

This flexible architecture allows organizations to tailor the Copilot backend deployment to their specific needs, balancing performance, security, and resource constraints while leveraging the power of containerization, vector databases, and large language models



2.1 Setup Contextual Documents

Contextual documents are additional pieces of text that you can provide to a language model like GPT to help it generate more accurate and context-aware responses. These documents can be used to provide background information, context, or specific details related to a conversation or task.

2.1.1 Setup Data Loaders

Data Loaders efficiently ingest various document types (e.g., PDF files, images) into Vector Databases (e.g., Milvus). The system offers multiple loader options based on the complexity of the algorithm used to extract different components of a document, such as text, images, tables, and metadata.

2.1.2 Available Loaders

To efficiently load various types of documents into Vector Databases, you can utilize one of the following Data Loaders based on your requirements:

1. Basic Text Loader:

- Purpose: Extracts plain text from documents
- Ideal for: Straightforward text ingestion without additional complexities
- Best used with: Text-based files, simple PDFs, or plain text documents.
- Features:
 - Fast processing of text-only content
 - Minimal resource requirements
 - Suitable for large volumes of simple documents

2. Advanced OCR-enabled Loader:

- Purpose: Comprehensive document analysis and extraction
- Features:
 - Optical Character Recognition (OCR) for image-heavy documents
 - Extracts text, images, and maintains document structure
 - Capable of identifying and extracting tables and structured data
- Ideal for: Complex documents, scanned files, or image-rich content
- Benefits:
 - High accuracy in extracting information from diverse documents types
 - Better preserving layout and formatting details

3. Custom Data Loaders:

- Purpose: Tailored extraction and processing for specific data sources or formats
- Options include:
 - Cloud Storage Loaders:

- AWS S3 Loader: Optimized for extracting data from S3 buckets
 - Azure Blob Storage Loader: Specialized for Azure Blob containers
 - Google Cloud Storage Loader: Designed for GCS buckets
- Database Loaders
 - SQL Database Loader: For extracting structured data from relational databases
- API Loaders
 - REST API Loader: For pulling data from web services
 - GraphQL Loader: Specialized for GraphQL endpoints
- Specialized Format Loaders:
 - XML/JSON Loader: For processing structured data files
 - CSV/Excel Loader: Optimized for tabular data
- Ideal For: Organizations with unique data sources or specific integration requirements
- Benefits:
 - Customized data extraction algorithms for optimal performance
 - Direct integration with existing data infrastructure
 - Support for proprietary or industry-specific data formats

Note: For assistance in developing custom data loaders or integrating specific data sources, please contact datascience@spotfire.com. Our team is ready to help you tailor solutions that meet your unique needs.

When selecting a data loader, consider factors such as document complexity, data source type, processing speed requirements, and the level of detail needed in the extracted information. Custom data loaders can be particularly beneficial for organizations with unique data ecosystems or those requiring specialized data processing pipelines.

2.1.3 Steps to set up Data Loaders:

1. Prepare Your Data:
 - a. For PDF files: Place them in a directory to be mounted to the Docker container.
 - b. For cloud-based documents:
 1. Azure Blobs: Identify the Azure Blob Storage container name.
 2. Amazon S3: Specify the S3 bucket name and, if applicable, the folder path within the bucket.
 3. Google Cloud Storage: Provide the GCS bucket name and optional object prefix.
 4. Dropbox: Determine the shared folder or file path.
 5. OneDrive: Identify the shared folder or file ID.
2. Access Data Stores:

- a. Ensure you have the necessary access credentials and permissions for the chosen cloud storage service. The specific details required may vary depending on the cloud provider and the Data Loader's configuration options.
3. Configure Docker Compose:
 - a. Select the appropriate docker-compose-<type>.yaml file from the appendix.
 - b. Customize environment variables in the selected file and the .env file.
 - c. Important: Specify the desired vector database and embedding model through plugin selection in the environment variables. For example:

None

```
VECTORDB_PLUGIN_ENTRY_POINT="plugins.vectordbs.zilliz:ZillizRetrieverPlugin
MODEL_PLUGIN_ENTRY_POINT="plugins.models.openai:OpenAIPlugin"
EMBEDDINGS_PLUGIN_ENTRY_POINT="plugins.embeddings.openai:OpenAIEmbeddingsPlugin"
```

Note: Refer to the appendix for sample Docker Compose files and a comprehensive list of required environment variables for each specific configuration. These samples provide templates for various setups, including different document types and data stores. Ensure you review and adjust the environment variables according to your chosen configuration before proceeding with the deployment.

By carefully selecting and configuring the appropriate Docker Compose file, you can tailor the setup to your specific needs, ensuring optimal performance and compatibility with your chosen vector database, language model, and embedding model

4. Deploy Docker Compose:
 - a. Execute the following command:

None

```
docker compose -f <chosen-file>.yaml up -d
```

5. Access the API:

The Data Loaders expose a comprehensive REST API, providing flexible access methods to suit various integration needs.

Note: This service uses Oath2-based authentication and acts as its own authentication server, allowing you to generate client_id and client_secret directly from the API. For detailed instructions on how to authenticate, register clients, and access secured endpoints, please refer to the **Authentication & Access Guide**.

Note on Available Operations

The API provides various operations, including:

- Loading custom documents
- Loading Spotfire documents
- Searching data stores

Refer to the full API documentation for detailed information on each endpoint and its usage.

6. Monitor and Verify:
 - a. Check Docker container logs for any issues.
 - b. Verify that documents are being processed and stored correctly.
7. Stop the application when finished:
 - a. Use the command: `docker compose -f <chosen-file>.yaml down`

By configuring the appropriate loader, vector database, and embedding model through environment variables, you can tailor the data ingestion process to your specific needs and document types. This flexibility allows for optimal extraction and storage of document components, enhancing the overall performance of your Copilot setup.

2.2 Setup Orchestrator

The Orchestrator is a pivotal component in facilitating rich interactions with Large Language Models (LLMs) like GPT. It acts as a dynamic bridge, fostering smooth and insightful exchanges between copilot users and these powerful models. The Orchestrator offers the following key capabilities:

- **Seamless Integration:** Connects with various data stores to provide LLMs with contextual data, enhancing response accuracy and relevance.
- **Conversation Management:** Maintains conversation history to ensure a continuous and coherent dialog flow.
- **Prompt Engineering:** Constructs and manages system prompts tailored to user intent.
- **Multi-Agent Collaboration:** Manages chains and orchestrates multiple AI agents to collaborate on complex use cases.
- **Extensibility:** Supports plugins for integrating additional LLMs and VectorDB retrievers.

2.2.1 Supported Integrations

- **LLM Platforms:** OpenAI, AzureOpenAI, Google Gemini, AWS Bedrock, Cohere, Hugging Face, Ollama, Nvidia NIMs
- **Vector Databases:** Milvus, Zilliz, Azure Cognitive Search, Redis, MongoDB, Chroma, Qdrant

2.2.2 Steps to set up the Orchestrator:

1. Access the Docker Compose Configuration:
 - a. Locate the docker-compose-orchestrator.yml file in the appendix.
 - b. Customize environment variables to align with your specific needs:
 - Specify plugins for extended functionality.
 - Configure LLM platforms and API settings.
 - Set up VectorDB connection parameters.
2. Prepare the Environment:
 - a. Copy docker-compose-orchestrator.yml and .env files to your desired directory.
3. Configure Docker Compose:
 - a. Select the docker-compose-orchestrator.yml file from the appendix.
 - b. Customize environment variables in the selected file and the .env file.
 - c. Important: Specify the desired vector database and LLM through plugin selection in the environment variables. For example:

None

```
RETRIEVER_PLUGIN_ENTRY_POINT="plugins.retrievers.zilliz:ZillizRetrieverPlugin"
MODEL_PLUGIN_ENTRY_POINT="plugins.models.openai:OpenAIPlugin"
EMBEDDINGS_PLUGIN_ENTRY_POINT="plugins.embeddings.openai:OpenAIEmbeddingsPlugin"
"
SECONDARY_MODEL_PLUGIN_ENTRY_POINT="plugins.models.openai:OpenAIPlugin"
```

Note: Refer to the appendix for sample Docker Compose files and a comprehensive list of required environment variables for each specific configuration. These samples provide templates for various setups, including different vector databases and model providers. Ensure you review and adjust the environment variables according to your chosen configuration before proceeding with the deployment.

By carefully selecting and configuring the appropriate plugins and environment variables, you can customize the Orchestrator to work seamlessly with your preferred vector database, large language model, and embedding model, maximizing its effectiveness for your use case.

4. Deploy Docker Compose:
 - a. Execute the following command:

None

```
docker compose -f docker-compose-orchestrator.yml up -d
```

5. Access the Orchestrator API:

The Orchestrator exposes a comprehensive REST API, providing flexible access methods to suit various integration needs.

Note: This service uses Oath2-based authentication and acts as its own authentication server, allowing you to generate `client_id` and `client_secret` directly from the API. For detailed instructions on how to authenticate, register clients, and access secured endpoints, please refer to the **Authentication & Access Guide**.

Refer to the full API documentation for detailed information on each endpoint and its usage.

6. Monitoring and Verification:

- a. Monitor Docker container logs for any issues or unexpected behavior.
- b. Verify that requests are handled appropriately and responses are as expected.

7. Stop the Application:

- a. When finished, stop the Orchestrator using:

None

```
docker compose -f docker-compose-orchestrator.yml down
```

By following these instructions, you'll set up a robust Orchestrator capable of interacting seamlessly with LLMs, managing contextual data, and supporting complex AI collaborations. The modular design allows for future extensibility, ensuring that your Copilot setup can adapt to new LLMs and data retrieval methods as they become available.

2.3 Authentication & Access Guide

Users can interact with the API through two primary approaches:

- Swagger UI Interface (Browser-based)
- Direct REST API Calls (Programmatic)

Both methods offer full access to the API's functionality, allowing users to leverage the api's capabilities in their preferred environment.

2.3.1 Swagger UI Interface

The Swagger UI offers an interactive, browser-based environment for exploring, testing, and interacting with the API endpoints. It allows you to authenticate, issue requests, view responses, and understand the structure of each operation—all without writing any code.

1. Open the API Docs
 - In your browser, navigate to:
`http://localhost:8080/docs`
 - This opens the Swagger UI, which provides an interactive interface to explore and test the API
2. Authenticate as Admin
 - Click the “**Authorize**” button at the top of the Swagger UI.
 - In the modal that appears, provide your admin credentials:
 - Username: admin
 - Password: Use the password you configured when generating the hashed password during setup.
 - Click “Authorize”.
 - A green lock icon indicates successful admin authentication.
Your admin token is now set and will be automatically included in all operations requiring it.
3. Register a new client
 - Call the **/register-client** endpoint to create a new client
 - This operation requires the admin token from Step 2.
 - The response will include a new generated:
 - client_id
 - client_secret
4. Get a Client Access Token
 - Use the **/client/token** endpoint to get an access token for the newly registered client.
 - Provide the client_id and client_secret
 - The response will include a Bearer token that grants access to all secured API operations
5. Authorize Secured Endpoints with Client Token
 - Click the swagger “**Authorize**” button again

- In the input field for HTTP Bearer, enter the token from Step 4.
 - Click Authorize to enable authentication calls for all secured endpoints
6. Using the API
- Browse available endpoints in the Swagger docs.
 - Click on any operation to view its details
 - Click “Try it out” to interactively test the operation
 - Provide required parameters and execute the call.
 - View the formatted response or error messages.

2.3.2 Direct REST API Access (Programmatic Integration)

To integrate this API into your applications or scripts, you can interact with it directly via HTTP using the OAuth2 client credentials flow. The API server acts as its own OAuth2 authorization server and provides endpoints to generate and use access tokens securely.

Base URL: <http://localhost:8080>

1. Authenticate as Admin and Get an Admin Token

Make a POST request to the /admin/token endpoint with admin credentials to obtain an admin access token.

Request:

```
None
POST /token
Content-Type: application/x-www-form-urlencoded

grant_type=password&username=admin&password=<your_admin_password>
```

Response:

```
None
{
  "access_token": "<admin_token>",
  "token_type": "bearer"
}
```

Use this access_token as the admin token in the next step

2. Register a New Client (Client ID & Secret)

Use the POST /register-client endpoint to generate a client_id and client_secret. This requires the admin token from the previous step.

Request:

```
None
POST /register-client
Authorization: Bearer <admin_token>
accept: application/json
```

Response:

```
None
{
  "client_id": "abc123",
  "client_secret": "s3cr3t456",
  "token_endpoint": "/client/token"
}
```

3. Get a Client Access Token

Use the POST /client/token endpoint to obtain an access token using the client credentials from Step 2.

Request:

```
None
POST /client/token
accept: application/json
Content-Type: application/x-www-form-urlencoded

'client_id=abc123&client_secret=s3cr3t456'
```

Response:

```
None
{
  "access_token": "client_token>",
  "token_type": "bearer"
}
```

4. Make Authenticated API Calls

Use the `client_token` from Step 3 to access any secured API endpoints.

Request:

None

`Authorization: Bearer <client_token>`

`accept: application/json`

5. General REST Usage Guidelines

- HTTP Methods: Use GET, POST, PUT, DELETE as defined in the API documentation
- Headers:
 - Authorization: Bearer <token> - Required for secured endpoints
 - Content-Type: application/json - For request with a body.
- Parameters: Include query parameters or JSON bodies as documented
- Responses: All responses are JSON-formatted.
- Status Codes:
 - 200 OK: Successful request
 - 401 Unauthorized: Invalid or missing token
 - 403 Forbidden: Permission denied
 - 400 Bad Request: Input Error
 - 500 Internal Server Error: Unexpected failure

APPENDIX

Docker Compose files

To utilize the Docker Compose files provided below, please ensure that you have Docker installed on your system. Follow these steps to setup the application:

1. Docker Compose file: Copy the corresponding Docker Compose content provided in the section below and save it as a .yml file.
2. Configuration: Open the saved Docker Compose file and review its contents. Make the necessary changes within the file. Ensure that environment variables, ports, volumes, database connection and API Keys and any service-specific configuration align with your setup
3. Launch the containers: Open a terminal or command prompt, navigate to the directory where the Docker Compose file is located, and execute the corresponding docker command.

Please note that you can provide environment variables to your Docker Compose service directly within the docker-compose-<>.yml file or by using a '.env' file in the same directory as your yml file.

Inside the '.env' file, define your environment variables in the format VAR_NAME=value. If using a .env file, reference these variables in the docker-compose-<>.yml file using the \${VAR_NAME} syntax.

Also note an issue related to Docker Compose when using environment variables that contain the "\$" character, especially when working with hashed passwords. If your hashed password contains a "\$", please replace each "\$" character with double "\$\$" to ensure that the password is correctly interpreted.

.env

None

LOG_LEVEL=INFO

AZSEARCH_EP="https://<changeme>.search.windows.net/"

AZSEARCH_KEY="changeme"

Data Loaders

Environment Variables

The table below offers a comprehensive description of all the necessary environment variables that might be required for the different data loaders. This information is essential for setting up and customizing the environment.

NAME	EXAMPLE VALUE	DESCRIPTION
LOG_LEVEL	INFO	Determines the verbosity of logging in the system, allowing users to control the amount of detail captured in log messages
Security Configuration Variables		
SECRET_KEY		Secret key that will be used to sign the JWT tokens.
HASHED_ADMIN_PASSWORD		Securely hashed password for the admin user, ensuring a protected and encrypted credential for administrator access in the system.
Plugin Configuration Variables		
VECTORDB_PLUGIN_ENTRY_POINT		Entry point for the vector database plugin. Currently supported plugins: plugins.vector dbs.zilliz:ZillizRetrieverPlugin plugins.vector dbs.milvus:MilvusRetrieverPlugin plugins.vector dbs.mongo:MongoRetrieverPlugin plugins.vector dbs.redis:RedisRetrieverPlugin plugins.vector dbs.az_cog_search:ACognitiveSearchRetrieverPlugin plugins.vector dbs.vertexai_vector_search:VertexAIVectorSearchRetrieverPlugin"
MODEL_PLUGIN_ENTRY_POINT		Entry point for the model plugin. Currently supported plugins: plugins.models.az_openai:AzOpenAIPlugin plugins.models.openai:OpenAIPlugin plugins.models.bedrock_anywhere:BedrockAnywherePlugin plugins.models.bedrock:BedrockPlugin plugins.models.cohere:CoherePlugin plugins.models.hugging_face_endpoint:HuggingFaceEndpointPlugin plugins.models.nvidia_nim:NvidiaNimPlugin plugins.models.ollama:OllamaPlugin plugins.models.vertexai:VertexAIPlugin
EMBEDDINGS_PLUGIN_ENTRY_POINT		Entry point for the embeddings plugin. Currently supported plugins: plugins.embeddings.openai:OpenAIEmbeddingsPlugin plugins.embeddings.az_openai:AzOpenAIEmbeddingsPlugin plugins.embeddings.hugging_face:HuggingFaceEmbeddingsPlugin plugins.embeddings.ollama:OllamaEmbeddingsPlugin plugins.embeddings.bedrock:BedrockEmbeddingsPlugin plugins.embeddings.bedrock_anywhere:BedrockAnywhereEmbeddingsPlugin plugins.embeddings.nvidia_nim:NvidiaNimEmbeddingsPlugin

		plugins.embeddings.vertexai:VertexAIEmbeddingsPlugin
Model Selection Variables		
MODEL_NAME		LLM Model name. Used when summarizing images
EMBEDDING_MODEL_NAME		Embedding model name
LLM Provider Connection Variables		
OpenAI		
OPENAI_API_TYPE	openai	Specifies the type of OpenAI API to be utilized, allowing users to configure and switch between different OpenAI API options in the system: azure or openai
OPENAI_API_KEY		Stores the authentication key required for accessing the OpenAI API, ensuring secure and authorized communication between the system and the OpenAI services.
OPENAI_API_VERSION	2023-05-15	Specifies the version of the OpenAI API to be used, allowing users to define and control the API version for seamless integration and compatibility in the system.
Azure OpenAI		
OPENAI_API_TYPE	azure	Specifies the type of OpenAI API to be utilized, allowing users to configure and switch between different OpenAI API options in the system: azure or openai
AZURE_OPENAI_ENDPOINT	https://<yourorg>.openai.azure.com	Defines the endpoint URL for the Azure OpenAI API, enabling users to set the API endpoint in the system and establish the connection to Azure OpenAI service. Not required if not using Azure OpenAI
OPENAI_API_KEY		Stores the authentication key required for accessing the OpenAI API, ensuring secure and authorized communication between the system and the OpenAI services.
OPENAI_API_VERSION	2023-05-15	Specifies the version of the OpenAI API to be used, allowing users to define and control the API version for seamless integration and compatibility in the system.
AWS Bedrock		
AWS_REGION		Sets the default AWS region for API requests.
AWS Anywhere Bedrock		
AWS_CONFIG_DIR		Specifies the directory path where AWS configuration files (config and credentials) are stored. This variable allows you to use a custom location for AWS configuration instead of the default ~/.aws directory.
AWS_PROFILE_NAME		Defines the name of the AWS profile to be used for authentication and configuration. This profile should be defined in the AWS credentials file and contains the necessary access keys and other settings.
AWS_REGION		Sets the default AWS region for API requests.
HuggingFace		

HUGGINGFACEHUB_URL		Specifies the base URL for accessing the Hugging Face Hub API.
HUGGINGFACEHUB_API_TOKEN		API token required for authenticating requests to the Hugging Face Hub.
Ollama		
OLLAMA_BASE_URL		Specifies the base URL for accessing the Ollama API
Google Vertex AI		
GOOGLE_APPLICATION_CREDENTIALS		Specifies the file path to a JSON file containing Google Credential to access Google API
Nvidia NIM		
NVIDIA_API_KEY		API key required for authenticating requests to NVIDIA's AI services,
NVIDIA_BASE_URL		Specifies the base URL for NVIDIA's AI service endpoints.
VectorDBs		
Azure Cognitive Search		
AZSEARCH_EP	https://<your endpoint name>.search.windows.net/	URL reference for connecting to the Azure Search service
AZSEARCH_KEY		Authentication key necessary for secure access to the Azure Search service
AZURE_STORAGE_CONNECTION	DefaultEndpointsProtocol=https;AccountName=<name>;AccountKey=<key>;EndpointSuffix=core.windows.net	Represents the connection string for Azure Storage, serving as a secure link to access and interact with Azure Storage service
AZURE_COGNITIVE_SEARCH_SERVICE_NAME		Identifies the name of the Azure Cognitive Search service
Milvus		
VECTORDB_URI		Specifies the base URL for Milvus's service endpoints.
VECTORDB_TOKEN		Authentication token for secure access to the Milvus VectorDB
Zilliz		
ZILLIZ_CLOUD_URI		Specifies the base URL for Zilliz's service endpoints.
ZILLIZ_CLOUD_API_KEY		Authentication token for secure access to the Zilliz VectorDB
Redis		
REDIS_URL		Specifies the connection URL for a Redis database instance. It contains all the necessary information to establish a connection to a Redis server, including the protocol, authentication credentials, host, port, and any additional connection parameters
MongoDB		

MONGODB_ATLAS_CLUSTER_URI		Specifies the connection string URI for your MongoDB Atlas cluster. It includes all necessary information to establish a connection, such as the username, password, cluster address, and any additional connection parameters.
MONGODB_ATLAS_DATABASE_NAME		Specifies the name of the database within your MongoDB Atlas cluster that you want to connect to and use for storing or retrieving data.
MONGODB_ATLAS_COLLECTION_NAME		Defines the name of the specific collection within the specified database where vector data or other relevant information will be stored or queried from.
MONGODB_ATLAS_INDEX_DIMENSIONS	1536	Specifies the number of dimensions for the vector index in MongoDB Atlas. This is crucial for vector search operations and determines the size of the vectors that can be stored and queried efficiently.
VertexAI Vector Search		
PROJECT_ID		The Google Cloud project ID where Vertex AI Search is configured and deployed.
LOCATION_ID		The Google Cloud region where Vertex AI Search services are hosted (e.g., us-central1).
GCS_BUCKET_NAME		The name of the Google Cloud Storage (GCS) bucket used to store search indexes or related assets for Vertex AI Search.
G_CLOUD_DIR		The directory path containing Google Cloud configuration credentials required for interacting with Vertex AI Search.
Volumes		
DOCS_DIR	./pdf_docs_folder	Directory path where PDF documents are stored, serving as a configuration parameter for the system to locate and load PDF files effectively.
Langsmith		
LANGCHAIN_API_KEY		Authentication key necessary to access Langsmith API
LANGCHAIN_TRACING_V2	true	Flag to enable/disable Langsmith monitoring
LANGCHAIN_ENDPOINT	https://api.smith.langchain.com	Defines the endpoint URL for Langsmith API
LANGCHAIN_PROJECT		Langsmith project name

Basic Data Loader - docker-compose-basic-data-loader.yml

None

```
services:
  data-loader:
    image: public.ecr.aws/tds/data-loader-pdf-pypdf:latest
    ports:
      - 8080:8080
    container_name: data-loader
    restart: unless-stopped
    volumes:
      - ${DOCS_DIR}:/docs
      - ${AWS_CONFIG_DIR}:/home/spotuser/.aws # Only for AWS Bedrock
    environment:
      LOG_LEVEL: ${LOG_LEVEL}
      SECRET_KEY: ${SECRET_KEY}
      HASHED_ADMIN_PASSWORD: ${HASHED_ADMIN_PASSWORD}
      VECTORDB_PLUGIN_ENTRY_POINT: ${VECTORDB_PLUGIN_ENTRY_POINT}
      MODEL_PLUGIN_ENTRY_POINT: ${MODEL_PLUGIN_ENTRY_POINT}
      EMBEDDINGS_PLUGIN_ENTRY_POINT: ${EMBEDDINGS_PLUGIN_ENTRY_POINT}
      MODEL_NAME: ${MODEL_NAME}
      EMBEDDING_MODEL_NAME: ${EMBEDDING_MODEL_NAME}
      # Model Provider Variables
      ...
      # Model Embedding Variables
      ...
      # VectorDB variables
      ...
```

Advanced Data Loader - public.ecr.aws/tds/data-loader-pdf-unstruct:latest

None

```
services:
  data-loader:
    image: public.ecr.aws/tds/data-loader-pdf-unstruct:latest
    ports:
      - 8080:8080
    container_name: data-loader
    restart: unless-stopped
    volumes:
      - ${DOCS_DIR}:/docs
      - ${AWS_CONFIG_DIR}:/home/spotuser/.aws # Only for AWS Bedrock
    environment:
      LOG_LEVEL: ${LOG_LEVEL}
      SECRET_KEY: ${SECRET_KEY}
      HASHED_ADMIN_PASSWORD: ${HASHED_ADMIN_PASSWORD}
      VECTORDB_PLUGIN_ENTRY_POINT: ${VECTORDB_PLUGIN_ENTRY_POINT}
      MODEL_PLUGIN_ENTRY_POINT: ${MODEL_PLUGIN_ENTRY_POINT}
      EMBEDDINGS_PLUGIN_ENTRY_POINT: ${EMBEDDINGS_PLUGIN_ENTRY_POINT}
      MODEL_NAME: ${MODEL_NAME}
      EMBEDDING_MODEL_NAME: ${EMBEDDING_MODEL_NAME}
      # Model Provider Variables
      ...
      # Model Embedding Variables
      ...
      # VectorDB variables
      ...
```

Orchestrator

Environment Variables

The table below offers a comprehensive description of all the necessary environment variables that might be required for the orchestrator. This information is essential for setting up and customizing the environment.

NAME	EXAMPLE VALUE	DESCRIPTION
LOG_LEVEL	INFO	Determines the verbosity of logging in the system, allowing users to control the amount of detail captured in log messages
Security Configuration Variables		
SECRET_KEY		Secret key that will be used to sign the JWT tokens.
HASHED_ADMIN_PASSWORD		Securely hashed password for the admin user, ensuring a protected and encrypted credential for administrator access in the system.
Plugin Configuration Variables		
RETRIEVER_PLUGIN_ENTRY_POINT		Entry point for the retriever plugin. Currently supported plugins: plugins.retrievers.milvus:MilvusRetrieverPlugin plugins.retrievers.zilliz:ZillizRetrieverPlugin plugins.retrievers.az_cog_search:AzCognitiveSearchRetrieverPlugin plugins.retrievers.mongo:MongoRetrieverPlugin plugins.retrievers.redis:RedisRetrieverPlugin plugins.retrievers.vertexai:VertexAIRetrieverPlugin
MODEL_PLUGIN_ENTRY_POINT		Entry point for the model plugin. Currently supported plugins: plugins.models.az_openai:AzOpenAIPlugin plugins.models.openai:OpenAIPlugin plugins.models.bedrock:BedrockPlugin plugins.models.bedrock_anywhere:BedrockAnywherePlugin plugins.models.cohere:CoherePlugin plugins.models.hugging_face_endpoint:HuggingFaceEndpointPlugin plugins.models.nvidia_mim:NvidiaNimPlugin plugins.models.ollama:OllamaPlugin plugins.models.vertexai:VertexAIPlugin
EMBEDDINGS_PLUGIN_ENTRY_POINT		Entry point for the embeddings plugin. Currently supported plugins: plugins.embeddings.ollama:OllamaEmbeddingsPlugin plugins.embeddings.openai:OpenAIEmbeddingsPlugin plugins.embeddings.az_openai:AzOpenAIEmbeddingsPlugin plugins.embeddings.hugging_face:HuggingFaceEmbeddingsPlugin plugins.embeddings.ollama:OllamaEmbeddingsPlugin plugins.embeddings.bedrock:BedrockEmbeddingsPlugin plugins.embeddings.bedrock_anywhere:BedrockAnywhereEmbeddingsPlugin plugins.embeddings.nvidia_nim:NvidiaNimEmbeddingsPI

		ugin plugins.embeddings.vertexai:VertexAIEmbeddingsPlugin
SECONDARY_MODEL_PLUGIN_ENTRY_POINT		Entry point for the secondary model plugin. Currently supported plugins: plugins.models.az_openai:AzOpenAIPlugin plugins.models.openai:OpenAIPlugin
Model Selection Variables		
MULTI_MODAL_MODEL_NAME	gpt-4o	Multi model name
COMPLETIONS_MODEL_NAME	gpt-3.5	Completions model name
CHAT_SIMPLE_MODEL_NAME	gpt-3.5-turbo-1106	Chat model name for simple scenarios
CHAT_COMPLEX_MODEL_NAME	gpt-4o	Chat model name for complex scenarios
EMBEDDING_MODEL_NAME	text-embedding-ada-002	Embedding model name
LLM Provider Connection Variables		
OpenAI		
OPENAI_API_TYPE	openai	Specifies the type of OpenAI API to be utilized, allowing users to configure and switch between different OpenAI API options in the system: azure or openai
OPENAI_API_KEY		Stores the authentication key required for accessing the OpenAI API, ensuring secure and authorized communication between the system and the OpenAI services.
OPENAI_API_VERSION	2023-05-15	Specifies the version of the OpenAI API to be used, allowing users to define and control the API version for seamless integration and compatibility in the system.
Azure OpenAI		
OPENAI_API_TYPE	azure	Specifies the type of OpenAI API to be utilized, allowing users to configure and switch between different OpenAI API options in the system: azure or openai
AZURE_OPENAI_ENDPOINT	https://<yourorg>.openai.azure.com	Defines the endpoint URL for the Azure OpenAI API, enabling users to set the API endpoint in the system and establish the connection to Azure OpenAI service. Not required if not using Azure OpenAI
OPENAI_API_KEY		Stores the authentication key required for accessing the OpenAI API, ensuring secure and authorized communication between the system and the OpenAI services.
OPENAI_API_VERSION	2023-05-15	Specifies the version of the OpenAI API to be used, allowing users to define and control the API version for seamless integration and compatibility in the system.
AWS Bedrock		
AWS_REGION		Sets the default AWS region for API requests.

AWS Anywhere Bedrock		
AWS_CONFIG_DIR		Specifies the directory path where AWS configuration files (config and credentials) are stored. This variable allows you to use a custom location for AWS configuration instead of the default ~/.aws directory.
AWS_PROFILE_NAME		Defines the name of the AWS profile to be used for authentication and configuration. This profile should be defined in the AWS credentials file and contains the necessary access keys and other settings.
AWS_REGION		Sets the default AWS region for API requests.
HuggingFace		
HUGGINGFACEHUB_URL		Specifies the base URL for accessing the Hugging Face Hub API.
HUGGINGFACEHUB_API_TOKEN		API token required for authenticating requests to the Hugging Face Hub.
Ollama		
OLLAMA_BASE_URL		Specifies the base URL for accessing the Ollama API
Google Vertex AI		
GOOGLE_APPLICATION_CREDENTIALS		Specifies the file path to a JSON file containing Google Credential to access Google API
Nvidia NIM		
NVIDIA_API_KEY		API key required for authenticating requests to NVIDIA's AI services,
NVIDIA_BASE_URL		Specifies the base URL for NVIDIA's AI service endpoints.
VectorDBs		
Azure Cognitive Search		
AZURE_COGNITIVE_SEARCH_SERVICE_NAME		Identifies the name of the Azure Cognitive Search service
AZURE_COGNITIVE_SEARCH_API_KEY		Authentication key required for secure access to the Azure Cognitive Search service
Milvus		
VECTORDB_URI		Specifies the base URL for Milvus's service endpoints.
VECTORDB_TOKEN		Authentication token for secure access to the Milvus VectorDB
Zilliz		
ZILLIZ_CLOUD_URI		Specifies the base URL for Zilliz's service endpoints.
ZILLIZ_CLOUD_API_KEY		Authentication token for secure access to the Zilliz VectorDB

Redis		
REDIS_URL		Specifies the connection URL for a Redis database instance. It contains all the necessary information to establish a connection to a Redis server, including the protocol, authentication credentials, host, port, and any additional connection parameters
MongoDB		
MONGODB_ATLAS_CLUSTER_URI		Specifies the connection string URI for your MongoDB Atlas cluster. It includes all necessary information to establish a connection, such as the username, password, cluster address, and any additional connection parameters.
MONGODB_ATLAS_DATABASE_NAME		Specifies the name of the database within your MongoDB Atlas cluster that you want to connect to and use for storing or retrieving data.
MONGODB_ATLAS_COLLECTION_NAME		Defines the name of the specific collection within the specified database where vector data or other relevant information will be stored or queried from.
MONGODB_ATLAS_INDEX_DIMENSIONS	1536	Specifies the number of dimensions for the vector index in MongoDB Atlas. This is crucial for vector search operations and determines the size of the vectors that can be stored and queried efficiently.
VertexAI Vector Search		
PROJECT_ID		The Google Cloud project ID where Vertex AI Search is configured and deployed.
LOCATION_ID		The Google Cloud region where Vertex AI Search services are hosted (e.g., us-central1).
GCS_BUCKET_NAME		The name of the Google Cloud Storage (GCS) bucket used to store search indexes or related assets for Vertex AI Search.
GLOUD_DIR		The directory path containing Google Cloud configuration credentials required for interacting with Vertex AI Search.
Langsmith		
LANGCHAIN_API_KEY		Authentication key necessary to access Langsmith API
LANGCHAIN_TRACING_V2	true	Flag to enable/disable Langsmith monitoring
LANGCHAIN_ENDPOINT	https://api.smith.langchain.com	Defines the endpoint URL for Langsmith API
LANGCHAIN_PROJECT		Langsmith project name

Orchestrator - docker-compose-orchestrator.yml

.env

None

```
services:
  orchestrator:
    image: public.ecr.aws/tds/llm-orchestrator:latest
    ports:
      - 8080:8080
    container_name: orchestrator
    restart: unless-stopped
    volumes:
      - ${AWS_CONFIG_DIR}:/home/spotuser/.aws # Only for AWS Bedrock
    environment:
      LOG_LEVEL: ${LOG_LEVEL}
      SECRET_KEY: ${SECRET_KEY}
      HASHED_ADMIN_PASSWORD: ${HASHED_ADMIN_PASSWORD}
      RETRIEVER_PLUGIN_ENTRY_POINT: ${RETRIEVER_PLUGIN_ENTRY_POINT}
      MODEL_PLUGIN_ENTRY_POINT: ${MODEL_PLUGIN_ENTRY_POINT}
      SECONDARY_MODEL_PLUGIN_ENTRY_POINT: ${SECONDARY_MODEL_PLUGIN_ENTRY_POINT}
      EMBEDDINGS_PLUGIN_ENTRY_POINT: ${EMBEDDINGS_PLUGIN_ENTRY_POINT}
      MULTI_MODAL_MODEL_NAME: ${MULTI_MODAL_MODEL_NAME}
      COMPLETIONS_MODEL_NAME: ${COMPLETIONS_MODEL_NAME}
      CHAT_SIMPLE_MODEL_NAME: ${CHAT_SIMPLE_MODEL_NAME}
      CHAT_COMPLEX_MODEL_NAME: ${CHAT_COMPLEX_MODEL_NAME}
      EMBEDDING_MODEL_NAME: ${EMBEDDING_MODEL_NAME}
      # Model Provider Variables
      ...
      # Model Embedding Variables
      ...
      # VectorDB variables
      ...
      LANGCHAIN_API_KEY: ${LANGCHAIN_API_KEY}
      LANGCHAIN_TRACING_V2: ${LANGCHAIN_TRACING_V2}
      LANGCHAIN_ENDPOINT: ${LANGCHAIN_ENDPOINT}
      LANGCHAIN_PROJECT: ${LANGCHAIN_PROJECT}
```

Docker Containers - Security Guidelines

Update Docker Regularly

Regularly update Docker to ensure that security patches are applied promptly.

Run Containers with Least Privileges

Run containers with the least necessary privileges. Avoid using the root user whenever possible.

Use Official Images

Prefer official Docker images from trusted sources to reduce the risk of vulnerabilities.

Container Isolation

Isolate containers from each other and the host system using features like namespaces and cgroups.

Network Security

Secure container communication by using Docker networks and implementing proper firewall rules.

Limit Resources

Control and limit container resources to prevent resource exhaustion attacks.

Monitoring and Logging

Implement monitoring and logging to detect and respond to security incidents.

Secure Docker Daemon

Secure the Docker daemon with TLS, restrict access, and enable authentication.

Regular Security Audits

Conduct regular security audits of your Docker environment to identify and address vulnerabilities.

Use the Latest Official Copilot Docker Images

Always deploy the latest official Copilot Docker images to ensure that critical security vulnerabilities and bugs are patched, and the most recent security enhancements are applied.